Data.Coerce: https://hackage.haskell.org/package/base/docs/Data-Coerce.html
Unsafe.Coerce: https://hackage.haskell.org/package/base/docs/Unsafe-Coerce.html

coerce
Converting between types with the same **runtime representation.**
In particular, automatically wrapping/unwrapping **newtype** layers

e.g. coerce newtype wrappers

```
import Data.Coerce
newtype A a = A a deriving Show
newtype B a = B a deriving Show

wrap :: A (B a) -> a
wrap = coerce

unwrap :: a -> (A (B a))
unwrap = coerce
```

e.g. coerce inner wrappers

```
import Data.Coerce

newtype A a = A a deriving Show

data Person a = Person { name:: n, age:: a }

wrap :: Person n a -> Person (A n) (A a)
wrap = coerce
```

Can use **unsafe coerce** if the compiler fails to automatically infer the coercion.
e.g. if the coercion is behind a type family layer

```
{-# LANGUAGE TypeFamilies #-}

import Data.Coerce
import Unsafe.Coerce

newtype Wrap1 a = Wrap1 a deriving (Show, Eq)

type family IdF a where
  IdF a = Wrap1 a

data WrapFamily a = WrapFamily (IdF a)

unsafeWrap :: WrapFamily Int -> WrapFamily (Wrap1 Int)
unsafeWrap = coerce            -- ERROR. "Couldn't match
representation of type 'Wrap1 Int' with that of 'Int'"
unsafeWrap = unsafeCoerce      -- OK
```